Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 · Unit 3: Functions, Functional Programming and Modules · List Comprehension

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · List Comprehension
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

List Comprehension

1 List Comprehension

Study focus: List Comprehension. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000046 · EV0000047

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Till now, we have understood how to create a list.

Key details

- A list can contain integers, it can be a float point, floating values, it can be strings, fine.
- In general, a list can be heterogeneous in nature.
- And this way of creating a list we call it as enumeration.
- And in today's class we will try to understand how to create list by using some rules.
- Creating a list using rules is called as list comprehension.
- There is also a dictionary comprehension, set comprehension, but they are all out of the scope for us so we are going to be learning only the concept of list comprehension.
- I in range.
- For I in range.
- while speaking I said it as five I in while I in range, but while typing I missed it.
- Can we really convert this into list comprehension?
- Can I write this using a rule?
- This is equal to.
- Because it's a list comprehension, I have to use a brackets here.
- Let me write for I in range.
- Now what is the next thing that I'll have to do?
- I'll have to print the values.
- Need what needs to be printed?
- It would have returned an object and I'll tell print this.
- The Python is printed for this.
- Let me comment on this.

- So now let me execute and no output because it has returned an object for us.
- We'll try to say print L1.
- Fine, and if you can see it has printed five times and within a list.
- And the general form.
- You have to write like this.
- General form is this.
- As usual, because it's a for loop variable.
- In eye trouble.
- for I variable in I triple.
- This is called as an expression.

Example or application from the lecture

- So just an example to show it to you, this is what we would have done.
- So let's execute this first and then we'll continue.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000046 · EV0000047 · EV0000048 · +1 source units in pack.json

Core explanation · 05:00–09:59

Execute the for a loop, fine, which is present as a part of list comprehension, execute this.

Key details

- Then every time you execute for loop, evaluate this expression.
- Execute for loops.
- Every time you execute a for loop, you will have to evaluate this particular expression.
- And thus the r resultant is a list that is getting created for us.
- This is an empty list, L1, that it's getting created at the beginning.
- I want to do square of a numbers.
- Square of numbers from.
- maybe one to ten?
- What do I need to do?
- Write a list comprehension, equivalentless comprehension.
- Because the moment I write level, it also includes a zero.
- And then do this.
- What I need to do?
- I need to perform the square of a number.
- And if you can see square of the numbers from one to ten has happened.
- I want to print.
- The string length.
- Along with that, I may also say Bangalore followed by the length.
- of strings and its length.
- [ASR low confidence; check audio].
- And I'll see PES University.
- Something of this kind.

- This is what I want to do and let me write a less comprehension.
- Or I in range.
- A for I in range.
- Sorry, not range.
- For I in A, because I already know I want to find out a string.
- and its length.
- It is available in I and I will say L of I.

Example or application from the lecture

- In the sense one to nine I want to do, do not consider ten.
- Or if you want to consider ten, you do it like this.
- Let's execute this and check.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000050 · EV0000051 · EV0000052 · +1 source units in pack.json

Core explanation · 10:00–14:59

Fine for I in a it should be fine.

Key details

- Execute this, I'm going to comment the next one also because I know what is the output here.
- Let me execute.
- Python six, PS University 14, language is eight, and so on.
- Please remember whenever you try to find the length, it also considers the space.
- I'll use the same list.
- And I want to convert all of them to uppercase.
- Fine, all of them to uppercase.
- I'll say L4 is equal to for INA because I'm still considering A itself.
- What do I need to do?
- I need to find the convert all of them to uppercase.
- You've already used this particular function in case of the strings request you to go back and check it out.
- There are so many things that you could do.
- You can also say that the one whose length is greater than six, print them and so on.
- I hope you have understood the concept of less comprehension.

Example or application from the lecture

- Let's use the same list.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000054 · EV0000055 · EV0000056 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
List Comprehension transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/009_List_Comprehension__27fdb808f3d32a2b.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
List Comprehension transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/009_List_Comprehension__27fdb808f3d32a2b.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
List Comprehension transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/009_List_Comprehension__27fdb808f3d32a2b.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.
List Comprehension transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/009_List_Comprehension__27fdb808f3d32a2b.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.

4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00018	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00019	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...
SRC00020	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00021	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...

C Revision and retrieval practice